

SISTEMA CLIENTE/SERVIDOR EM JAVA

Relatório Técnico do Projeto

Engenharia Informática | 2025/2026 | 1º Ano
Redes de Computadores | Professor Fernando Moreira

Ana Souto | 53986

António Santos | 47303

Diogo Ferreira | 53501

João Pinto | 53255

Tomás Santos | 54379



ÍNDICE

Descrição Geral da Aplicação	4
Arquitetura do Sistema	4
Organização dos módulos principais	4
Protocolo de Comunicação	5
Explicação do uso de TCP	7
Explicação do uso de UDP	7
Estratégia de Concorrência	8
Concorrência no Servidor	8
Concorrência no Cliente	9
Estratégia de Validação	9
Validação de registo/autentificação	9
Validação de comandos.....	10
Validação de ficheiros.....	10
Estratégia de Timeouts.....	10
Servidor TCP: Prevenção de Sessões Inativas	11
Cliente UDP: Gestão de Falhas de Entrega	11
Plano de testes.....	11

Principais Dificuldades Encontradas	12
Conclusão	13
ANEXO 1	14
ANEXO 2.....	15

Descrição Geral da Aplicação

A aplicação desenvolvida consiste num sistema distribuído de comunicação em tempo real e partilha de ficheiros baseado na arquitetura cliente/servidor, recorrendo à linguagem Java.

O objetivo central do projeto é demonstrar a aplicação prática dos conceitos de programação em sockets, controlo de concorrência, desenho de protocolos de aplicação, delimitação de mensagens (framing), codificação de caracteres (encoding) e tratamento de falhas na rede através de interrupções por timeouts.

O sistema é composto por duas entidades principais: o Servidor e o Cliente. Para a comunicação, o sistema utiliza uma combinação dos protocolos TCP e UDP, tirando partido das características de cada um para satisfazer os requisitos do sistema.

Arquitetura do Sistema

A aplicação utiliza uma arquitetura Cliente/Servidor, com o código organizado em três módulos distintos (server, client e udp) para separar responsabilidades lógicas e facilitar a manutenção do sistema.

Organização dos módulos principais

A lógica do sistema está dividida da seguinte forma:

- **Server:** A classe Servidor inicia o serviço TCP e gere o seu funcionamento principal. Cada cliente que se liga é entregue à classe Atendente, responsável pela comunicação com esse cliente. A classe GerirCliente mantém a informação dos utilizadores ligados, enquanto a classe Logger regista os eventos do sistema.
- **Client:** A classe Cliente gere a interação com o utilizador e o envio de comandos para o servidor. Existe uma classe auxiliar ThreadRecetor, cuja única função é escutar continuamente as respostas enviadas pelo servidor.

- **UDP:** As classes `UdpClienteDescoberta` e `UdpServidorDescoberta` implementam um sistema que utiliza datagramas para localizar automaticamente o endereço IP e a porta do servidor na rede local.

Protocolo de Comunicação

O protocolo de aplicação define as regras para a troca de dados entre o Cliente e o Servidor. Para garantir uma fiabilidade e evitar a corrupção do fluxo contínuo do TCP, o protocolo foi estruturado com base nas regras de formatação (Framing) e codificação (Encoding).

A comunicação no nosso sistema divide-se em dois formatos distintos, dependendo do tipo de dados a transmitir:

- **Codificação Universal:** Todas as mensagens de texto trocadas são forçadas bilateralmente a utilizar a codificação UTF-8. Isto garante que os caracteres são traduzidos corretamente entre sistemas operativos diferentes, prevenindo a receção de caracteres ilegíveis.
- **Interação em Texto:** Para os comandos escritos pelo utilizador e para as respostas do servidor, foi utilizada a estratégia de delimitadores. O fim de cada mensagem é assinalado pelo carácter de quebra de linha `\n`. Isto permite que a leitura seja feita de forma clara através de métodos como o `readLine()`.
- **Ficheiros binários (como imagens ou PDFs)** não podem usar caracteres delimitadores, como o utilizado `\n`, para indicar o fim do envio, e como esses bytes podem existir naturalmente no meio do conteúdo do ficheiro, é necessário aplicar o padrão Length-Prefix para o comando. Com este padrão, o recetor sabe exatamente quantos bytes deve ler, porque a informação é enviada seguindo esta estrutura rigorosa:
 - **Tamanho do nome (4 bytes - Int):** Indica o comprimento do nome do ficheiro.
 - **Nome do ficheiro (Bytes):** O nome real do ficheiro, codificado em UTF-8.
 - **Tamanho do conteúdo (8 bytes - Long):** Indica o tamanho total do ficheiro em bytes.
 - **Conteúdo do ficheiro (Bytes):** Os dados reais do ficheiro, transmitidos progressivamente em pacotes através de um ciclo.

Para o cliente interagir com o servidor, ele pode utilizar comandos de texto pré-definidos como:

Comando	Descrição
NICK <nome>	Regista a identidade do utilizador (Obrigatório ao iniciar).
HELP	Mostra a lista de comandos disponíveis no sistema.
WHO	Solicita a lista com os nomes de todos os utilizadores ligados.
MSG <texto>	Envia uma mensagem pública em broadcast para a sala.
PM <nick> <texto>	Envia uma mensagem privada para um utilizador específico.
SEND <ficheiro>	Inicia a transferência de um ficheiro até 5 MB para o servidor.
QUIT	Termina a sessão do utilizador e fecha a ligação de forma limpa.
DISCOVER	Procura o servidor na rede local através de broadcast.

De forma semelhante ao protocolo HTTP, o servidor valida todas as ações e responde com um código numérico de estado seguido de uma mensagem legível, permitindo ao cliente saber se a operação teve sucesso ou se gerou um erro:

Código	Significado	Exemplo de Mensagem
200	Sucesso: A operação foi processada corretamente.	200 bem-vindo joao
400	Erro de Formato: O comando ou os dados são inválidos	400 ficheiro demasiado grande
401	Não Autorizado: O cliente tentou usar comandos sem se registar.	401 Cliente ainda não registado
404	Não Encontrado: O recurso ou utilizador destino não existe.	404 utilizador não encontrado
408	Timeout: A ligação expirou por inatividade prolongada.	408 timeout
409	Conflito: O recurso (como o nome de utilizador) já está em uso.	409 nome já utilizado

Explicação do uso de TCP

O TCP é responsável por toda a comunicação essencial: a autenticação, o envio de comandos, o chat e a transferência de ficheiros. A utilização deste protocolo garante três características fundamentais:

- **Garantia de entrega e ordem:** O TCP oferece um stream fiável e ordenado. Ao contrário do UDP, o TCP assegura que os pacotes chegam ao destino na mesma ordem em que foram enviados. Esta característica mostrou-se vital para a transferência de ficheiros, uma vez que um byte desordenado pode levar a corrupção do ficheiro inteiro, e para a leitura por Framing.
- **Orientação à ligação (Stateful):** Por exigir um estabelecimento de ligação prévio, com o método 3-way handshake, o TCP mantém um canal aberto e contínuo. Isto é essencial para a funcionalidade de chat, pois permite ao servidor saber de forma precisa quais os clientes que estão ativamente ligados na sessão num específico momento.
- **Controlo de fluxo e congestionamento:** Durante a transferência de um ficheiro de 5 MB, o TCP gere automaticamente a velocidade de transmissão de bytes, evitando sobrecarregar o recetor ou a rede.

Explicação do uso de UDP

No nosso projeto, o UDP foi utilizado exclusivamente para a implementação da funcionalidade complementar do sistema: descobrir o servidor disponível na rede local. A escolha do UDP para esta tarefa garante as seguintes vantagens:

- **Capacidade de difusão (Broadcast):** Sendo um protocolo sem ligação, o UDP permite enviar um datagrama, no nosso caso o comando DISCOVER, para o endereço global da sub-rede (255.255.255.255). O TCP é estritamente ponto-a-ponto e nunca permitiria procurar por um servidor sem saber previamente o seu IP.
- **Comunicação imediata:** O UDP não exige o processo demorado de criar uma ligação. O pacote é simplesmente lançado para a rede, tornando o processo de descoberta instantâneo.

- **Gestão de perdas com timeouts:** Uma vez que o UDP não garante a entrega da mensagem, o protocolo de aplicação foi desenhado para apoiar essa falha. O cliente define um timeout de 3000ms à espera da resposta. Se o pacote DISCOVER for perdido ou não existir nenhum servidor na rede, o cliente não fica bloqueado a espera de resposta; simplesmente levanta uma exceção de timeout e avisa o utilizador.

Estratégia de Concorrência

Para garantir que a aplicação seja capaz atender vários clientes em simultâneo sem perda de desempenho ou bloqueio de recursos, o sistema adota dois modelos de concorrência distintos, adaptados às necessidades do Servidor e do Cliente.

Concorrência no Servidor

O servidor foi desenhado para nunca bloquear a sua execução principal. Para isso, utiliza duas estratégias principais:

- **Delegação de Tarefas (ExecutorService):** O servidor recorre a uma thread pool através do `Executors.newCachedThreadPool()`. Sempre que um novo cliente estabelece ligação, a gestão dessa sessão (Atendente) é entregue a uma thread da pool. Desta forma, a thread principal do servidor fica imediatamente disponível para continuar a aceitar novas ligações TCP.
- **Estruturas de Dados Seguras (Thread-Safe):** Como múltiplas threads (correspondentes a diferentes clientes) podem tentar entrar na sala de chat, sair ou enviar mensagens ao mesmo tempo, a lista global de utilizadores gerida pela classe `GerirCliente` é implementada com recurso a um **ConcurrentHashMap**. Esta estrutura da biblioteca padrão do Java evita race conditions e corrupção nos dados durante acessos simultâneos, garantindo a integridade da informação sem necessitar de locks (bloqueios manuais).

Concorrência no Cliente

Se o cliente operasse numa única thread, a operação de leitura do teclado bloquearia a receção de mensagens da rede. Para garantir uma experiência em tempo real, o cliente separa as operações de Entrada/Saída (I/O):

- **Thread Principal:** Dedicada exclusivamente à captura dos comandos do utilizador e ao envio das mensagens para o servidor.
- **Thread Secundária (ThreadRecetor):** Executada em paralelo, tem como função "escutar" continuamente o canal de rede e imprimir no ecrã as mensagens recebidas, sem interferir com o processo de escrita do utilizador.

Estratégia de Validação

A robustez de um sistema distribuído depende da premissa defensiva de que "não se deve confiar nos dados enviados pela rede". Desta forma, o servidor implementa um conjunto rigoroso de validações antes de processar qualquer pedido:

Validação de registo/autentificação

Quando o cliente tenta definir o seu identificador, na utilização do comando NICK, o servidor verifica o seguinte:

- A String não é nula composto apenas por espaços em branco.
- O tamanho obedece aos limites estipulados, mínimo de 3 máximo de 20 caracteres.
- O nome escolhido já não esta a ser utilizado por outro cliente.

Validação de comandos

Os comandos de texto pré-definidos pelo sistema quando são introduzidos pelo cliente, também sofrem validações.

- Verifica se o comando pertence à lista de comandos disponíveis ao cliente. Caso ele envie um comando que não pertence é retornado 400 comando invalido.
- Se existem argumentos em falta no comando, por exemplo, se um cliente apenas introduzir “PM” sem introduzir um destinatário ou texto o servidor garante que esses campos têm de ser preenchidos.
- O servidor ignora inputs acidentais, isto é, introduções resultantes do input da tecla Enter.

Validação de ficheiros

Durante a transferência de dados pelo comando SEND, aplicam-se algumas proteções de validação:

- Prevenção de Path Transversal: Ficheiros cujo o nome contenham caracteres relativos a diretório, como são os exemplos “..”, “/” ou “\”, são imediatamente bloqueados para evitar que um utilizador com intuítos maliciosos escreva ficheiros fora da pasta de uploads permitida.
- Para prevenir ataques de estresse de armazenamento, o servidor verifica o tamanho declarado do ficheiro antes de o receber, sendo rejeitado se este estiver vazio (0 bytes) ou se tiver um tamanho demasiado grande (superior a 5 MB).

Estratégia de Timeouts

Em programação de redes, funções como `readLine()` ou `receive()` são operações que aguardam resposta. Se um cliente perder a ligação subitamente, o sistema operativo não envia um aviso imediato, o que pode deixar as threads bloqueadas indefinidamente, acabando por consumir recursos do sistema de forma desnecessária.

Para garantir a estabilidade, implementámos timeouts estruturais. Quando o tempo limite é atingido, o sistema lança uma `SocketTimeoutException`, permitindo que a aplicação recupere o controlo e depois tome uma decisão. Esta estratégia foi aplicada em dois níveis, sendo elas:

Servidor TCP: Prevenção de Sessões Inativas

No servidor, aplicamos um timeout de 3 minutos a cada nova ligação (socket.setTimeout(180000)). Isto evita a acumulação de threads inativas na pool caso um cliente deixe de comunicar sem fechar a sessão. Se o tempo expirar, o servidor captura a exceção, envia a mensagem de estado 408 timeout, regista o evento e encerra a ligação de forma segura, libertando os recursos associados.

Cliente UDP: Gestão de Falhas de Entrega

Como o UDP não garante a entrega, o UdpClienteDescoberta utiliza um timeout curto de 3 segundos (socketUDP.setTimeout(3000)) ao emitir o comando de broadcast "DISCOVER". Caso não receba resposta, a exceção é tratada internamente e o utilizador é informado de que nenhum servidor foi localizado, permitindo-lhe optar pela introdução manual do endereço IP.

Plano de testes

Nº	Teste Realizado	Resultado Confirmado
1	Cliente liga ao servidor TCP	Servidor aceita ligação e cria sessão.
2	Cliente usa WHO antes de NICK	Servidor responde 401.
3	Cliente regista nome válido	Servidor responde 200.
4	Dois clientes tentam usar o mesmo nome	O segundo cliente recebe 409.
5	Cliente envia MSG	Outros clientes recebem a mensagem.
6	Cliente envia PM para utilizador ativo	Destinatário recebe a mensagem.
7	Cliente envia PM para utilizador inexistente	Servidor responde 404.
8	Cliente envia comando inválido	Servidor responde 400.
9	Cliente envia ficheiro válido (SEND)	Servidor guarda o ficheiro.
10	Cliente envia ficheiro acima do limite	Servidor rejeita (Erro).
11	Cliente tenta enviar ../x.txt	Servidor rejeita (Erro).
12	Cliente fica inativo (Silêncio na rede)	<i>Timeout</i> é tratado.
13	Dois ou mais clientes comunicam em simultâneo	Servidor continua funcional.
14	Cliente fecha inesperadamente	Servidor remove o cliente.
15	Cliente emite descoberta UDP (DISCOVER)	Recebe resposta ou <i>timeout</i> controlado.

Principais Dificuldades Encontradas

As principais dificuldades enfrentadas pelo grupo durante o desenvolvimento do projeto dividiram-se entre a vertente técnica e a gestão de tempo.

A nível técnico, o maior desafio foi, unanimemente, a compreensão e aplicação do código a programar. Este domínio revelou-se complexo, mas o mesmo era essencial não só para dar resposta a todos os requisitos solicitados, tal como para a implementação de funcionalidades adicionais.

Paralelamente, o desenvolvimento deste trabalho coincidiu com a execução de outro projeto igualmente exigente, sendo que o mesmo carecia de bastante tempo e atenção ao detalhe. Esta sobreposição de prazos dificultou a divisão de esforços do grupo, tornando-se um desafio conciliar a dedicação a ambos para conseguir garantir, como produto final, dois projetos rigorosos e bem executados.

Conclusão

O desenvolvimento deste projeto revelou-se uma experiência importante para consolidar na prática os conceitos teóricos da Unidade Curricular Redes de Computadores. Criar um sistema cliente/servidor do zero obrigou-nos a sair da teoria e a lidar com problemas reais, como falhas de rede, mensagens partidas e a dificuldade de pôr um servidor a atender várias pessoas ao mesmo tempo sem bloquear.

Durante a implementação, percebemos na prática por que motivo usamos diferentes protocolos. O TCP foi fundamental para garantir que os ficheiros e as mensagens de *chat* não se perdiam nem chegavam corrompidos, enquanto o UDP se mostrou a ferramenta certa para descobrirmos rapidamente o servidor na rede local de forma simples. Além disso, a necessidade de usar timeouts e gerir as threads fez-nos entender que um bom programa não pode apenas funcionar quando tudo corre bem; tem de saber lidar com erros e com clientes que se desligam do nada.

Como referimos nas dificuldades, a sobreposição com outros trabalhos e a compreensão do código deram-nos bastantes dores de cabeça. Ainda assim, com esforço e divisão de tarefas, conseguimos ultrapassar esses obstáculos e entregar uma aplicação funcional que cumpre os objetivos propostos. Saímos deste projeto com uma noção muito mais clara e realista de como as máquinas comunicam entre si.

ANEXO 1

Declaração de Autoria – para Trabalho de Grupo

João Pinto, Ana Souto, António Santos, Diogo Ferreira e Tomás Santos, estudantes do curso de Licenciatura de Engenharia Informática, declaram que o Relatório apresentado para avaliação é da nossa autoria e cumpre as normas de integridade académica.

Assim, comprometemo-nos a:

- Esclarecer, explicitamente, se partes do trabalho foram já apresentadas para avaliação de outras UC ou provas de grau na Universidade Portucalense ou noutras instituições;
- Identificar corretamente as fontes que utilizamos, de forma que possam ser consultadas e atestada a autenticidade do trabalho que apresentamos;
- Assumir, sob compromisso de honra, a responsabilidade da autoria integral do trabalho, não tendo contratado serviços de terceiros para a sua realização;
- Indicar a supervisão que recebi para a elaboração do trabalho;
- Reconhecer como fraudulentas as práticas que correspondem a formas de plágio, cópia servil, omissão ou citação deficiente de fontes, percebendo que tais práticas infringem direitos de autoria e são contrárias à integridade académica;
- Submeter, quando solicitado, à consideração do(s) docente(s), relatórios que tenham sido emitidos por equipamento especializado na deteção de plágio.

07/06/2026

Assinaturas

J. Pinto  António Santos Diogo Ferreira 

ANEXO 2

Declaração sobre o uso de IA Generativa para a realização de trabalhos académicos

Curso/UC: Engenharia Informática

Ano letivo: 2025/2026

Trabalho: Sistema Cliente/Servidor em Java

IDENTIFICAÇÃO DOS ESTUDANTES

João Pinto (53255), Ana Souto (53986), António Santos (47303), Diogo Ferreira (53501) e Tomás Santos (54379)

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Neste trabalho usámos sistemas de Inteligência Artificial (IA) Generativa para melhorar a linguagem/estrutura do texto e pesquisar informação.

Os sistemas de IA Generativa utilizados neste trabalho foram os seguintes: Gemini e Claude.

ÂMBITO DE UTILIZAÇÃO

Pesquisar informação, esclarecer dúvidas sobre código e sobre teoria, correção de erros

PROMPTS UTILIZADOS

“Explica o que é (conceito teórico)”; “Como resolvo este erro?”; “Como posso implementar (x)?”; “Melhora o seguinte parágrafo”

07/06/2026





UNIVERSIDADE
PORTUCALENSE

upt.pt